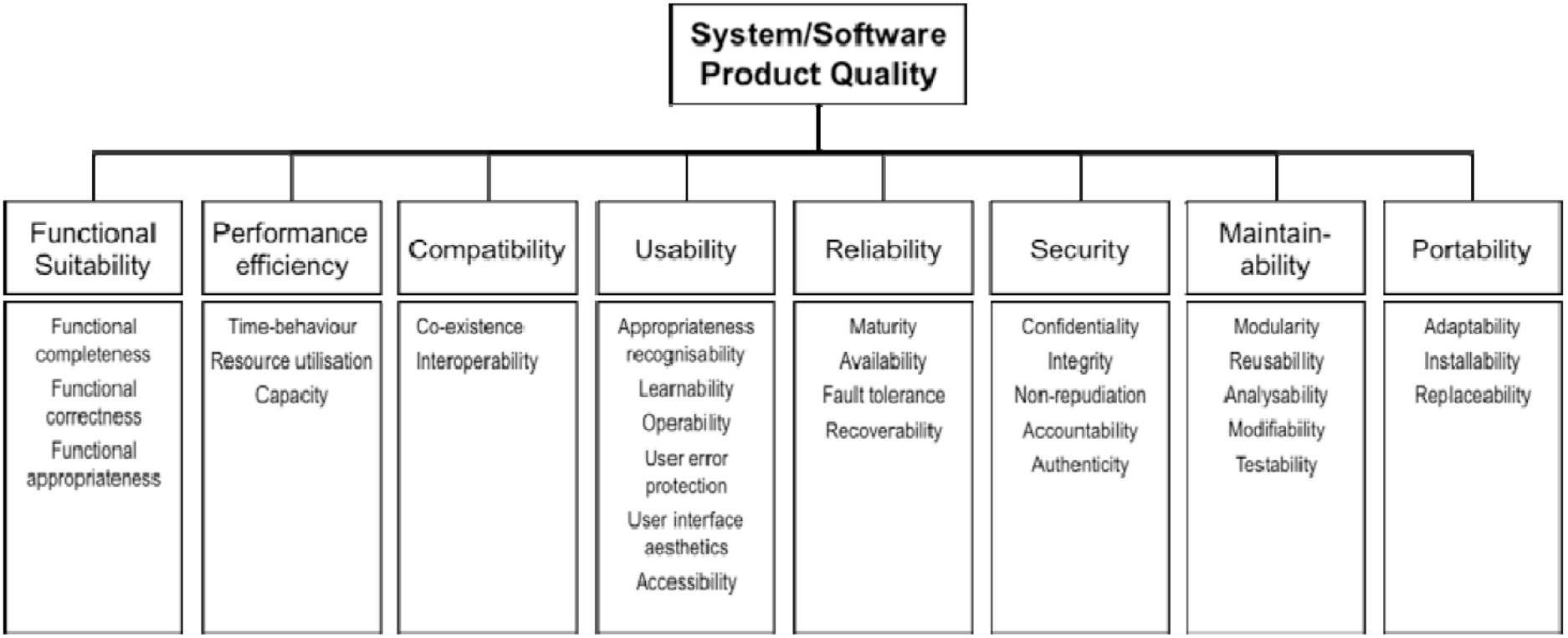
QUALITÀ DEI SISTEMI SOFTWARE

Michele Marchesi

marchesi@unica.it

A.A. 2025-26

I criteri di qualità secondo ISO-IEC 25010



La norma ISO-IEC 25010 supera la norma ISO 9126

1. Idoneità Funzionale

- Un programma, o la implementazione di un sistema, è **funzionalmente idoneo** se si comporta sempre come definito nella specifica dei requisiti funzionali.
- Definizione di Idoneità Funzionale:
  "*Il grado in cui un prodotto o sistema fornisce funzioni che soddisfano bisogni dichiarati e impliciti quando utilizzato in condizioni specificate*"
- L'idoneità funzionale è suddivisa in 3 livelli
  - completezza funzionale
  - correttezza funzionale
  - appropriatezza funzionale

1. Idoneità Funzionale

- **Completezza funzionale**: tutti i requisiti sono coperti e implementati
- **Correttezza funzionale**: i requisiti sono implementati correttamente (funzionano come specificato)
- **Appropriatezza funzionale**: i requisiti sono implementati in modo efficace per il contesto di uso
- Sono verificate con metodi sperimentali (testing) o analitici (formal verification)

2. Efficienza (efficiency, performance)

- Un sistema è **efficiente** se usa le sue risorse in maniera economica. Tipicamente: usa poco **spazio** (disco, memoria centrale) e poco **tempo** per fornire i propri servizi.
- Questa proprietà migliora in genere col tempo, col miglioramento della tecnologia hardware (ci sono sempre più memoria e velocità).
- Per alcuni sistemi (real-time, sistemi Web massivi…) è una **qualità essenziale** (fa parte della correttezza funzionale)

2. Efficienza

- L’efficienza si declina nelle seguenti caratteristiche:
  - **Comportamento temporale**: il tempo impiegato dal sistema a rispondere
  - **Utilizzo delle risorse**: il consumo di CPU/memoria/banda di rete
  - **Capacità**: quantità di utenti e/o dati gestibili dal sistema
- L'efficienza è sempre relativa a condizioni specificate (carico, hardware, contesto operativo).
- La valutazione delle prestazioni è fatta tramite:
  - analisi di complessità degli algoritmi (caso medio, caso peggiore)
  - analisi matematica e simulazione sul modello.
  - misura sul campo (alla fine, è quello che conta!)

Scalabilità

- Si ha scalabilità quando il sistema può “crescere” e “decrescere” a seconda delle necessità
- Ha a che fare con Capacità e Utilizzo delle risorse
- Crescita:
  - capacità di usare più memoria e più potenza di calcolo senza modifiche al codice
  - capacità di utilizzare più macchine/CPU che operano in parallelo, in numero che dipende dalle richieste
    - il sistema deve essere progettato per poterlo fare

3. Compatibilità - Interoperabilità

- **Coesistenza**: capacità di operare insieme ad altri sistemi.
- **Interoperabilità**: capacità di scambiare dati con altri sistemi.
- Es.: cooperazione fra le diverse componenti (Foglio elettronico, Word processor, Posta, Slide) di pacchetti software integrati.
- Viene ottenuta standardizzando le interfacce (**XML, Json**).
- Sempre più importante con l'avvento delle reti e di Internet (tutto comunica con tutto!)

4. Usabilità (usability, user-friendliness)

- *Grado in cui un prodotto o sistema può essere utilizzato da utenti specificati per raggiungere obiettivi specificati con efficacia, efficienza e soddisfazione, in un contesto d'uso specificato*
- Si divide in:
  - **Ricognizione di appropriatezza**: L'utente capisce che il sistema è adatto al suo bisogno
  - **Apprendibilità**: serve poco tempo per imparare a usare le funzioni del sistema
  - **Operabilità**: facilità di controllo e uso del sistema
  - **Protezione dagli errori**: il sistema previene e aiuta a recuperare gli errori
  - **Estetica**: l'interfaccia è piacevole e supporta l'engagement
  - **Accessibilità**: il sistema è utilizzabile da persone disabili

4. Usabilità

- **Documentazione** e interfaccia utente (*user-interface, UI*) *appropriate*
  - Proprietà soggettiva. Es.
    - Spiegazioni verbose e menù sono apprezzati dall'utente inesperto, non dall'esperto.
  - Proprietà relativa: i sistemi sono più facili da usare se hanno UI simili a quelle delle soluzioni più diffuse
    - **Mac --> Windows --> app mobili**
    - **In futuro…?**
  - In sistemi senza interfaccia utente (embedded): facilità di configurazione e adattamento all'ambiente hardware.
  - **Ricerca su nuovi paradigmi di interazione uomo-macchina**

Accessibilità

- *Grado in cui prodotti, sistemi e ambienti possono essere utilizzati da persone con il più ampio spettro di caratteristiche e capacità*
- Tipi di disabilità:
  - Visiva → tipo di carattere, dimensione, contrasto…
  - Uditiva → alternative testuali per contenuti audio
  - Motoria → navigazione da tastiera, tempi di risposta estendibili, comandi vocali
  - Cognitiva → linguaggio semplice, layout prevedibile
- Esempio: "*Tutte le pagine dell'intranet aziendale devono: (1) essere navigabili interamente da tastiera, (2) avere contrasto testo/sfondo ≥4.5:1, (3) fornire alternative testuali per immagini informative, (4) essere testate con screen reader (NVDA/ VoiceOver) senza perdita di funzionalità.*"

5. Affidabilità (reliability)

- *Grado in cui un sistema, prodotto o componente esegue le funzioni richieste in condizioni specificate per un periodo di tempo specificato*
- Si divide in:
  - **Maturità**: Frequenza dei guasti durante l'operatività normale.
  - **Disponibilità** (Availability): Grado di operatività quando richiesto. Es. uptime 99.99%.
  - **Tolleranza ai guasti** (Fault Tolerance): Mantenimento delle prestazioni in caso di guasto hardware/software, o degradazione gentile.
  - **Ripristinabilità** (Recoverability): Capacità di recuperare dati e stato dopo un'interruzione.

5. Affidabilità

- Proprietà statistica, espressa in termini della **probabilità** che il sistema operi come previsto in un determinato intervallo temporale.
  - **Non implica correttezza**: un programma può essere scorretto (non soddisfa le specifiche del cliente), ma affidabile (opera sempre)
  - **Tasso di errore** (**Failure rate**): misura o stima il numero di fallimenti per unità di tempo, o per n. di transazioni o di esecuzioni del programma
- **Disponibilità** del sistema o dei dati (percentuale di tempo in cui il sistema è *up*, ovvero disponibile):
  - Es: up per il 99,5% del tempo
  - Sistemi 24/7: devono essere up sempre!

Sicurezza (per sistemi "safety critical")

- Sicurezza assoluta di funzionamento nel caso di sistemi in cui il fallimento software può causare perdite umane
- Esistono precise regolamentazioni internazionali sullo sviluppo di software *life critical* nei vari settori (aeronautico, ferroviario, medicale, ecc.)
- Implica spesso revisioni e certificazioni ottenute da enti di controllo esterni
- Consiste in *correttezza* e *affidabilità* portate al massimo livello!

Robustezza

- Capacità del sistema di gestire input invalidi, condizioni al limite o stress ambientali senza collassare.
- Comportamento ‘ragionevole’ anche in casi non previsti nei requisiti (es. dati input scorretti o errore del disco).
  - Es: il sistema non perde i propri dati nemmeno se viene spento improvvisamente durante il funzionamento
- Se il problema è previsto nei requisiti, gestirlo correttamente rientra nella correttezza
- Ma i requisiti non possono specificare tutti i casi anomali!

6. Protezione (Security)

- *Grado in cui un prodotto o sistema protegge informazioni e dati in modo che persone, prodotti o sistemi con un tipo di autorizzazione e un livello di autorizzazione specifici possano accedervi, nonché in modo che possano essere utilizzati da altri prodotti o sistemi*
- Si divide in:
  - **Riservatezza** (Confidentiality): I dati sono accessibili solo a chi è autorizzato
  - **Integrità**: I dati possono essere alterati solo se ciò è autorizzato
  - **Non ripudio** (Non-repudiation): Un utente non può negare di aver compiuto un'azione
  - **Responsabilità** (Accountability): Tutte le azioni sono tracciabili a un'identità specifica
  - **Autenticità**: L'identità di utenti/sistemi è verificabile

6. Protezione (Security)

- **È particolarmente critica** per sistemi in rete e su Internet
- Comprende la protezione da minacce informatiche, quali **virus**, **worm**, **trojan**, ecc.
- Comprende la protezione da **operazioni illegittime effettuate da utenti e sviluppatori con diritti di accesso**
  - Sono il vero punto debole dei sistemi
  - Soggetti a corruzione o ricatto
- Distinzione Critica: Security vs. Safety:
  - SECURITY (Protezione): Difesa da minacce INTENZIONALI
  - SAFETY (Sicurezza operativa): Prevenzione da danni NON INTENZIONALI

7. Manutenibilità - evolvibilità

- *Grado in cui un prodotto o sistema può essere modificato in modo efficace ed efficiente da manutentori specificati per correggere difetti, soddisfare nuovi requisiti, facilitare la manutenzione futura o adattarsi a un ambiente modificato*
- Si divide in:
  - **Modularità**: Il sistema è composto da componenti a basso accoppiamento.
  - **Riusabilità**: Le componenti possono essere riutilizzate in altri contesti.
  - **Analizzabilità**: È facile diagnosticare difetti o identificare parti da modificare.
  - **Modificabilità**: Le modifiche possono essere fatte senza introdurre nuovi difetti.
  - **Testabilità**: Il sistema facilita la verifica dei criteri di accettazione.

7. Manutenibilità - evolvibilità

- La manutenibilità non è solo un attributo del codice, ma del sistema socio-tecnico: include codice, documentazione, processi e competenze del team.
- Deve essere possibile correggere e far evolvere il software per riflettere nuovi requisiti
  - Ma i cambiamenti devono interessare tutta la pila di documenti del processo!
- Si ottiene con una progettazione modulare e col "**refactoring**"
- Tanto più importante quanto più utilizzato e di successo è il sistema software!

Riusabilità

- *Grado in cui un manufatto può essere utilizzato in più contesti o sistemi*
- Simile a **evolvibilità**, ma riferito a nuovi prodotti, anziché a versioni dello stesso prodotto.
- Applicabile soprattutto a parti del sistema.
- Librerie FORTRAN, C, Java, Python disponibili sul mercato.
- Proprietà ben realizzata con l'approccio ad oggetti per librerie di "basso livello" (widget, interfaccia verso il DBMS…)
- Molto difficile da realizzare in modo economico e pratico per componenti di alto livello

Verificabilità

- *Grado in cui i requisiti e i manufatti del sistema possono essere valutati in modo oggettivo rispetto a criteri di accettazione definiti*
- La specifica, il progetto, il codice vengono realizzati in modo da semplificarne la verifica di proprietà:
  - Certi linguaggi sono più verificabili di altri.
  - Il codice può venire arricchito con '*software mo-ni-tors*': sezioni che, se una flag è attivata, stampano lo stato delle variabili su un 'log'
  - E' aiutata dalla *tracciabilità* dei requisiti nei documenti di progetto e nel codice

8. Portabilità

- *Grado in cui un sistema, prodotto o componente può essere trasferito in modo efficace ed efficiente da un ambiente hardware, software o operativo a un altro*
- Si divide in:
  - **Adattabilità**: Il sistema può essere configurato per ambienti diversi senza modifiche al codice.
  - **Installabilità**: È efficiente installare/disinstallare il sistema in un ambiente target.
  - **Sostituibilità** (Replaceability): Il prodotto può sostituire un altro software con le stesse finalità.
  - **Co-esistenza** / **Compatibilità**: Il sistema opera correttamente condividendo risorse con altri prodotti.

8. Portabilità

- Quando l'applicazione può girare in diversi ambienti (piattaforma HW o SW)
- Si può ottenere separando nettamente le funzionalità indipendenti dalla piattaforma da quelle dipendenti
  - In tale caso, basta specializzare per ogni sistema solo lo strato dipendente dalla piattaforma
- Inoltre, il sistema deve utilizzare solo un sottoinsieme *stabile* di risorse dell'ambiente, o di istruzioni macchina.

Proprietà interne ed esterne del software

- Si distinguono proprietà:
  - **Esterne / In Uso**: quelle visibili durante i test di sistema e dall'utente finale.
  - **Interne**: quelle che riguardano chi lo sviluppa, lo mantiene, lo cambia
  - Le interne aiutano a ottenere le esterne: esempio, la verificabilità aiuta a ottenere la affidabilità.
- L'importanza di una data proprietà dipende dal tipo di prodotto e dal suo ambiente di utilizzo.
  - In sistemi safety-critical e real-time, affidabilità, sicurezza e efficienza sono cruciali

Riassunto delle proprietà del software

- **Esterne**
  - Correttezza
  - Affidabilità
  - Sicurezza
  - Protezione
  - Efficienza
  - Usabilità
  - Robustezza
  - Interoperabilità
  - Portabilità
- **Interne**
  - Manutenibilità
  - Verificabilità
  - Riusabilità
  - Scalabilità
  - Interoperabilità
  - Portabilità

Conflitti tra proprietà del software

- Non si possono ottenere tutte le qualità definite da ISO/IEC 25010 al massimo grado!
- Esempi:
  - **Sicurezza vs. Usabilità**: Misure di sicurezza stringenti (autenticazione MF, password complesse, timeout brevi) aumentano il carico cognitivo e il numero di passaggi per l'utente, riducendo l'usabilità.
  - **Efficienza vs. Manutenibilità**: Ottimizzazioni aggressive (cache manuali, query SQL denormalizzate, codice low-level) migliorano performance ma rendono il codice complesso, accoppiato e difficile da modificare.
  - **Affidabilità/Disponibilità vs. Costo/Efficienza**: Aumentare la disponibilità (es. da 99.9% a 99.99%) richiede ridondanza attiva, failover automatico e infrastruttura duplicata, con maggiori costi e maggiore consumo di risorse.

Requisiti di qualità in diverse aree applicative

- Sistemi Informativi
- Sistemi Real-time
- Sistemi Distribuiti
- Sistemi Embedded
- Sistemi Web
- Apps
- Dapps

Sistemi Informativi

- Sistemi **orientati ai dati**. La loro funzione è gestire informazioni ed effettuare elaborazioni sui dati. Alcuni sono detti **sistemi transazionali**
  - Costruiti attorno a un *data base* che viene manipolato mediante *transazioni* (C*reate, Retrieve, Update, Delete* – ***CRUD***).
  - *Es.: banca, biblioteca, gestione personale, contabilità, magazzino, comando e controllo, simulazione.*
- **Integrità dei dati (robustezza)**
  - I dati non devono perdersi o diventare scorretti per malfunzionamenti del sistema
- **Protezione**
  - Protezione da accessi non autorizzati
- **Disponibilità dei dati (affidabilità)**
  - Quando e per quanto tempo i dati diventano inaccessibili?
- **Prestazioni (efficienza)**
  - Transazioni per unità di tempo.

Sistemi Real-time

- Sistemi **orientati al controllo**
  - Devono rispondere a eventi esterni entro tempi prefissati, spesso (non sempre!) brevissimi.
  - **Es1.** Sistema controllo edificio: aumento temperatura → allarme (*OK anche se ritarda di qualche sec.*)
  - **Es2.** Controllo torri di cracking di una raffineria: se certe reazioni durano troppo (anche troppi decimi di secondo), la torre può esplodere!
- Tempi di risposta
  - Sono relativi all'efficienza in sistemi generici, ma alla correttezza in sistemi real-time
- Affidabilità e correttezza sono essenziali!

Sistemi Distribuiti

- Insieme di computer collegati da **rete (nodi + connessioni)**
  - Banda larga e basso tasso di errori di trasmissione → possibilità di distribuire le componenti del sistema.
  - Distribuzione di dati e/o funzioni
- Sistemi tipicamente **Robusti**
  - **Tolleranza** a fallimenti dei *nodi*
  - **Tolleranza** a fallimenti delle *connessioni*
- Rendere distribuito un sistema può migliorare alcune sue *proprietà*:
  - Replicazione dei dati → *affidabilità, robustezza*
  - Distribuzione dei calcoli → *efficienza*

Sistemi Embedded

- La componente software è ***incastonata*** *in uno specifico hardware (non un PC)* e lo controlla.
  - *Es. Aereo, frigorifero, ABS, robot, pacemaker*
- Necessarie varie qualità:
  - Efficienza, vista la limitata potenza dell'hw
  - Affidabilità, sicurezza per sistemi safety-critical
- Requisiti meno stringenti per l'**usabilità**.
  - Ma non se la UI è presente ma con hw limitato (ad es., una lavatrice o un forno a microonde)
- Il Sistema è più **evolvibile** se si spostano funzionalità da HW a SW.

Sistemi Web

- Sistemi che operano in Internet, protocolli http e https
- Intrinsecamente distribuiti
- Accesso e GUI tramite Web Browser
- I siti più popolari possono avere decine di migliaia di accessi al secondo!
- Necessarie molte qualità:
  - Efficienza e scalabilità, per i siti più popolari
  - Protezione da accessi non autorizzati (hacker)
  - Usabilità
  - Portabilità, per quanto riguarda i browser
- Requisiti meno stringenti per **sicurezza** e **interoperabilità**

“Apps”

- Sono le applicazioni per terminali mobili (Smartphone e Tablet PC)
- Utilizzano quasi sempre l'accesso alla rete Internet
- Utilizzano funzionalità specifiche del terminale (GPS, accelerometri, *touch*, ...)
- Necessarie varie qualità:
  - Efficienza, visto anche la limitata potenza di alcuni TM
  - Protezione da accessi non autorizzati
  - Usabilità
  - Portabilità, ottenuta tramite lo standard HTML5
- Sono spesso applicazioni relativamente piccole (ma sempre meno, vista la crescente potenza dell'hw)

Dapps – Smart Contracts

- Dapps (Decentralized apps): sono i programmi che girano su Blockchain (Smart Contracts)
- Sono attivati da transazioni, e **non possono accedere a risorse esterne** (file, rete, dispositivi di input...)
- Necessarie varie qualità:
  - Correttezza e affidabilità, anche perché **non sono modificabili**, una volta rilasciati entro la blockchain.
  - Efficienza, poiché girano su macchine virtuali nei nodi della blockchain, e spesso hanno costi sia di esecuzione che di memorizzazione (GAS)
  - **Estrema necessità di sicurezza e protezione**, poiché spesso gestiscono grandi somme di criptovaluta o NFT
- Sono tipicamente applicazioni relativamente piccole